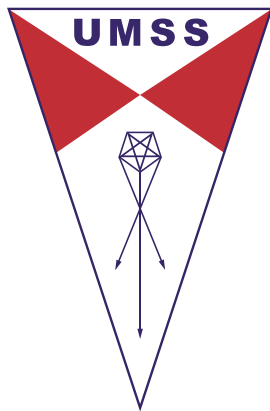


Analisis de Cabezeras

Con WireShark



Jhosua Alejandro Bustillos Calderon

Carrera
Ingeniería Informatica

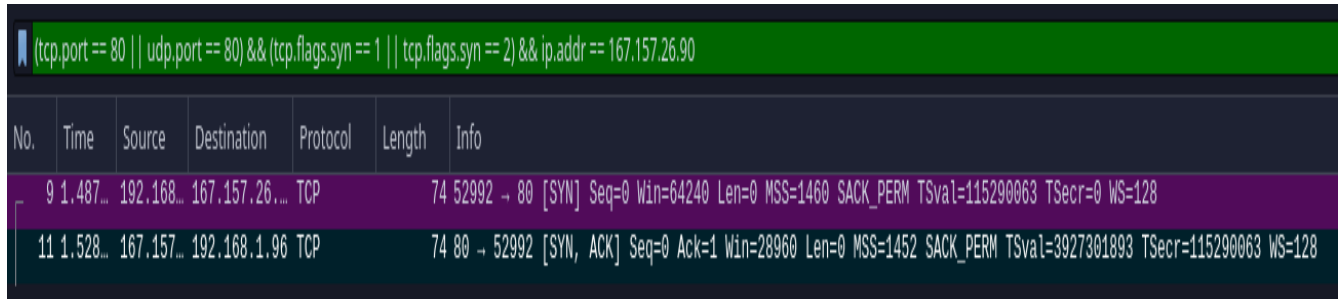
Materia
Programacion Web

Docente
Ing. COSTAS JAUREGUI VLADIMIR ABEL

2 de septiembre de 2026

1. Reporte <http://www.cs.umss.edu.bo/>

Usa el filtro: (tcp.port == 80 || udp.port == 80) && tcp.flags.syn == 1 && ip.addr == 167.157.26.90
Analiza el handshake y la cabecera para SYN y ACK del TCP.



No.	Time	Source	Destination	Protocol	Length	Info
9	1.487...	192.168...	167.157.26...	TCP	74	52992 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=115290063 TSecr=0 WS=128
11	1.528...	167.157...	192.168.1.96	TCP	74	80 → 52992 [SYN, ACK] Seq=0 Ack=1 Win=28960 Len=0 MSS=1452 SACK_PERM TSval=3927301893 TSecr=115290063 WS=128

Figura 1: Captura de paquetes

1.1. Analisis

Se observa que el handshake inicializa en el frame 9(SYN) y continua en el frame 11(SYN, ACK), faltaria el ACK que devuelve el servidor al cliente. El ACK se encuentra en el frame 12. El cliente inicia la conexion y se le otorga un numero secuencial Seq = 0, tambien se encuentran atributos como: tamaño de la shell, el segmento maximo(tamaño de datos), entre otros.

Posteriormente, el servidor confirma el SYN del cliente guardandolo en ACK = 1 y tambien otorga su propio numero secuencial Seq = 0. Ambos frames pesan lo mismo.

Finalmente el cliente confirma el SYN del servidor y se sincroniza la comunicacion como establecida entre ambos. Este ultimo frame pesa menos porque ya no hace la negociacion de las opciones del cliente.

1.2. Prompt

¿Que sucede en el frame 12, que datos se pierden/recuperan o se omiten?

2. Reporte www.google.com

Analiza la cabecera de UDP para el DNS usa el filtro: dns.qry.name == "www.google.com"previo a la captura de paquetes en linux aplica este comando en la terminal: sudo killall -HUP mDNSResponder

NOTA: en Debian, no existe el proceso mDNSResponder, por lo que me fui al propio google con chrome://net-internals/#dns para limpiar la caché de DNS. Debian resuelve DNS directamente contra 8.8.8.8/8.8.4.4 sin caché de sistema.

No.	Time	Source	Destination	Protocol	Length	Info
5	0.889...	192.168...	192.168.1.1	DNS	74	Standard query 0x3a77 HTTPS www.google.com
6	0.889...	192.168...	192.168.1.1	DNS	74	Standard query 0x4768 AAAA www.google.com
7	0.889...	192.168...	192.168.1.1	DNS	74	Standard query 0xf64b A www.google.com
8	0.894...	192.168...	192.168.1.96	DNS	298	Standard query response 0x4768 AAAA www.google.com AAAA 2001:4860:4826:7700:: AAAA 2001:4860:482a
9	0.895...	192.168...	192.168.1.96	DNS	202	Standard query response 0xf64b A www.google.com A 142.251.154.119 A 142.251.157.119 A 142.251.151
14	0.902...	192.168...	192.168.1.96	DNS	451	Standard query response 0x3a77 HTTPS www.google.com HTTPS A 142.251.151.119 A 142.251.157.119 A 1

Figura 2: Captura de paquetes

2.1. Analisis

En el frame 7, esta el paquete donde mi maquina pregunta que IPv4 tiene google, la cabecera de UDP, carag con lo siguiente: el port que eligio Debian para hacer la consulta, el port de DNS, el tamaño contando cabecera y el payload más el checksum.

El payload es donde vive el mensaje completo DNS como ser, la cabecera de DNS(transacciones, flags, preguntas),el nombre codificado de google.

En el frame 9, esta el paquete donde google responde con más de una IP siendo estos multiples registros. La cabecera de UDP contiene el port desde donde responde (53) y el port destino, el tamaño en bytes y el checksum. El payload contiene practicamente lo mismo que el anterior paquete junto al bloque de respuestas.

2.2. Prompt

¿Que contiene el payload del UDP de cada frame?(Descripcion adjunta de los frames)

3. Reporte rtp_example.raw

Para RTP por UDP analiza una de las cabeceras. Carga en wireshark el archivo: rtp_example.raw (adjunto)

Para el analisis debes elegir el paquete a analizar , fijate en los encabezados de TCP/UDP.

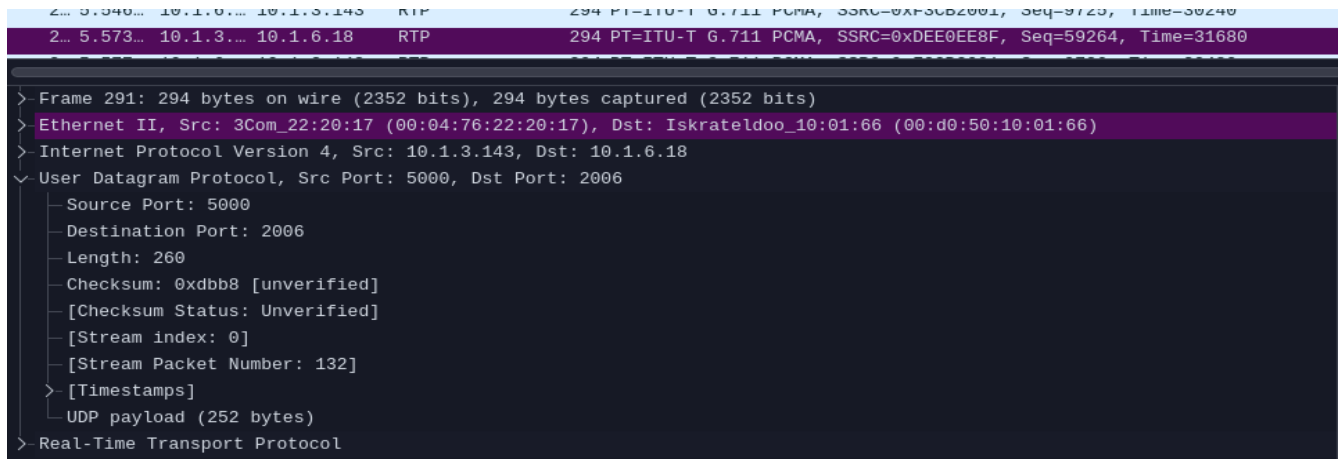


Figura 3: Captura de paquete elegido

3.1. Analisis

Se tiene el frame 291, en sí todo el archivo contiene una llamada, para eso son los protocolos RTP y H.#. En la cabecera de UDP, se encuentra lo siguiente:

- Port de emisor
- Port de destino
- El tamaño del paquete
- El checksum

Pero lo genial es la cabecera del RTP ya que contiene el payload del UDP, en este caso se tiene contenido de la llamada, analizando el CS: setup en el frame la IP 10.1.3.143 es la que inicia la llamada, entonces en el frame 291 se tiene lo que contesta este mismo cliente.